

Retrieval Eval: BM25 Tokenization Fix and Experiment Attribution

KernelPack RAG — Experiments v1–v3

May 2026

Codebase: kernelpack-python @ b0cc141 **Eval set:** qa_pairs/solvers_qa.json — 10 pairs, kernelpack.solvers, tiers: api (4), workflow (3), conceptual (3).

Note: QA pairs are Codex-generated and not yet validated by a domain expert. Recall numbers are directional only.

Background

Two notebooks cover this experiment series. They run identical pipeline logic and eval sets, differing in exactly one place: BM25 tokenization.

02a_eval_bm25_mangling.ipynb uses whitespace-only tokenization (`.split()`), which treats `PoissonSolver.solve` as a single indivisible token. It is preserved as a research artifact so the BM25 contribution can be isolated by comparison with the fixed version.

02b_eval_baseline.ipynb uses identifier-aware tokenization (`bm25_tokenize`), splitting on camel-Case and snake_case boundaries so `PoissonSolver.solve` becomes `["Poisson", "Solver", "solve"]`. This is the authoritative notebook.

What the broken tokenizer actually does

BM25 matches query tokens against indexed chunk tokens. With whitespace-only tokenization, a query like *“how does VariablePoissonSolver solve a system?”* produces the compound token `VariablePoissonSolver`, which never appears verbatim in the query text. BM25 scores every chunk near-zero. The sparse leg of the hybrid pipeline is functionally dead — retrieval degrades to dense-only with RRF overhead.

Recall@3 Results

02a — broken BM25 (whitespace tokenization)

Version	Change	recall@3
v1	full index, raw source, MIN_LINES=5	1/10 (10%)
v2	MIN_LINES=1	1/10 (10%)
v3a	solvers only, no summaries (control)	2/10 (20%)
v3	solvers only, LLM summaries prepended	4/10 (40%)

02b — fixed BM25 (camelCase-aware tokenization)

Version	Change	recall@3
v1	full index, raw source, MIN_LINES=5	5/10 (50%)
v2	MIN_LINES=1	4/10 (40%)
v3a	solvers only, no summaries (control)	5/10 (50%)
v3	solvers only, LLM summaries prepended	8/10 (80%)

Attribution

BM25 fix: +4 at the v1 baseline. Comparing v1 across notebooks: 1/10 (broken) vs. 5/10 (fixed). The four recovered targets are ones where the query’s natural-language phrasing matches identifier tokens once they are split. This is the single largest gain in the experiment series and required no pipeline change.

LLM summaries (v3): +3 on top of the fixed BM25. In 02b: v3a = 5/10, v3 = 8/10. Summaries close the vocabulary gap between natural-language queries and code identifiers. `pu_patch_weight` is the clearest example: dense rank 29 in v1, dense rank 1 in v3, after a plain-English summary gives the embedding signal to match against the query. In 02a, summaries add only +2 over the same control (2→4), because the broken BM25 cannot leverage the enriched text.

Scope reduction (v3a): 0 contribution. In both notebooks, v3a matches v1 exactly at recall@3. Narrowing the index to `kernelpack.solvers` improves ranks in the wide diagnostic but does not move any buried target across the recall@3 threshold. The control experiment cleanly isolates this: all of v3’s gain is attributable to summaries, not scoping.

MIN_LINES=1 (v2): negative. In 02b, v2 = 4/10, down from 5/10. Expanding from 337 to 614 chunks by lowering the line-count threshold introduces short accessor functions with noisy embeddings. `VariablePoissonSolver.solve` drops out of the top-50 dense window entirely under v2. No not-indexed failures existed in v1 to recover, so the change had no upside.

Wide Retrieval Diagnostic (02b, v3, top-50)

Dense ranks isolate embedding quality. Hybrid ranks show BM25+RRF on top. The delta between columns is the BM25 contribution per target.

Target	v1 dense	v1 hybrid	v3 dense	v3 hybrid	Category
<code>PoissonSolver.solve</code>	18	2	6	2	Recovered hit
<code>VariablePoissonSolver.solve</code>	41	8	18	6	Reranker problem
<code>NonlinearVariablePoissonSolver.solve</code>	47	6	18	3*	Eval miss
<code>PUSLAdvectionSolver.forward_sl_step</code>	3	2	4	1	Hit
<code>DiffusionSolver.set_state_history</code>	1	1	4	3	Hit
<code>MultiSpeciesDiffusionSolver.set_initial_state</code>	4	4	2	2	Recovered hit
<code>PUSLFDAdvectionDiffusionSolver.bdf3_step</code>	2	1	6	3	Hit
<code>build_stencil_properties</code>	2	1	1	1	Hit
<code>pu_patch_weight</code>	29	13	1	1	Recovered hit
<code>IncompressibleEulerBDFBackend._get_cached_system</code>	15	5	7	3	Recovered hit

*The wide diagnostic uses a top-50 candidate pool; the recall@3 eval uses top-10 dense + top-10 BM25. `NonlinearVariablePoissonSolver.solve` reaches hybrid rank 3 in the expanded pool but not the eval window — these numbers are not contradictory. This target was a v1 hit carried entirely by BM25 (dense rank 47, hybrid rank 6). Under the solvers-only scope in v3, BM25 signal weakened and it regressed. Root cause TBD before v4.

Key Findings

1. **The vocabulary gap is closed after v3.** All 10 targets appear in the top-50 dense neighborhood. Recall@5 in v3 = 10/10 — a reranker window of 5 has a theoretical ceiling of 10/10 at recall@3.

2. **The bottleneck is ranking, not coverage.** The two remaining recall@3 misses (`VariablePoissonSolver.solve` and `NonlinearVariablePoissonSolver.solve`, both at dense rank 18) are ranking failures. A cross-encoder reranker is the highest-leverage next step.
3. **BM25 tokenization is not a detail.** A 40-point recall gap at the v1 baseline traces to one incorrect function call. Identifier-aware tokenization is the mechanism by which the sparse leg matches function names to natural-language queries.
4. **MIN_LINES is the wrong filter.** It excludes real targets, degrades recall when lowered, and has no principled relationship to chunk quality. Replace with a content-aware filter in v4.

Next Step

Cross-encoder reranker on the v3 index. Rerank the top-30–50 hybrid candidates by scoring each query-chunk pair directly. Starting point: `cross-encoder/ms-marco-MiniLM-L-6-v2`. Run the same 10-query eval before and after; the delta is the reranker’s isolated contribution.

Resolve the `NonlinearVariablePoissonSolver.solve` regression before implementing v4.